

Чек-лист приемки, релиза и рисков для mock-django

Эксплуатационная готовность CI/CD-развертывания в контуре ЦБС

Назначение документа

Этот PDF фиксирует приемочные критерии, эксплуатационный чек-лист, процедуру релиза и риски для внедрения CI/CD-развертывания mock-django в контуре ЦБС Петроградского района.

Приемочный чек-лист перед первым staging-деплойм

Блок	Проверка	Ожидаемый результат
Repository	main protected, MR required, CODEOWNERS/ответственные назначены.	Изменения проходят review, прямой push в main закрыт.
Secrets	SECRET_KEY, DB password, SSH keys отсутствуют в Git; .env не коммитится.	Секреты хранятся в GitLab variables или на сервере.
Django	python manage.py check --deploy проходит с production settings.	Нет критических предупреждений deployment checklist.
Tests	Есть тесты для index и /health/; CI запускает manage.py test.	Pipeline падает при ошибке тестов.
Image	Docker image собирается и push в GitLab registry по SHA tag.	Образ доступен для docker compose pull.
Compose	db healthcheck и web healthcheck настроены; restart policy включена.	web не стартует до готовности БД; /health/ отвечает.
Server	Docker/Compose установлены, deploy-user создан, firewall/TLS готовы.	Деплой возможен без ручного root-доступа.
Backup	pg_dump/retention/restore rehearsal описаны.	Перед production релизом есть свежий backup и проверка restore.

Процедура релиза

- 1 Создать MR с изменениями deployment-пакета и application hardening; добиться прохождения lint/test/build.
- 2 Смержить MR в main: staging pipeline должен автоматически собрать image и развернуть staging.
- 3 Проверить staging: главная страница, /health/, docker compose ps, последние логи web/db, миграции.
- 4 Создать release tag vYYYY.MM.DD-N или vX.Y.Z; открыть production job manual в GitLab.
- 5 Перед запуском production job убедиться, что есть актуальный backup БД и rollback image tag.
- 6 Запустить deploy:production, проверить health URL и smoke tests. Зафиксировать результат в релизном журнале.
- 7 После релиза 30-60 минут наблюдать логи, ошибки 5xx, доступность сервиса и состояние БД.

Процедура отката

Сценарий	Действия	Критерий успеха
Ошибка приложения после deploy	Установить APP_IMAGE на предыдущий SHA/tag; docker compose pull; docker compose up -d; проверить /health/.	Старый образ запущен, /health/ 200.
Неуспешные миграции	Остановить rollout, восстановить БД из backup при необходимости, запустить предыдущий image.	Данные согласованы, приложение отвечает.
Недоступен reverse proxy/TLS	Проверить nginx-proxy/certs/firewall; при необходимости временно открыть внутренний порт только для диагностики.	Домен снова отвечает по HTTPS.
Проблема с registry	Использовать уже загруженный local image на сервере либо повторить docker login/pull после восстановления registry.	Контейнеры подняты из доступного image.

Риск-регистр

ID	Риск	Вероятность/влияние	Меры снижения
R1	Содержимое исходного GitLab CI недоступно как текст, перенос может отличаться от оригинальной реализации.	Средняя/среднее	Использовать публично подтвержденную схему GitLab CI/CD + registry + Docker Compose; после получения доступа сверить .gitlab-ci.yml и обновить шаблоны.
R2	Hard-coded SECRET_KEY и wildcard ALLOWED_HOSTS в mock-django создают security debt.	Высокая/высокое	До production вынести SECRET_KEY/ALLOWED_HOSTS в env, включить проверку check --deploy, запретить секреты в Git.
R3	PostgreSQL в compose без backup/restore rehearsal.	Средняя/высокое	Настроить регулярный pg_dump, retention, тест восстановления, регламент перед релизом.
R4	Деплой по SSH может получить слишком широкие права.	Средняя/среднее	Отдельный deploy-user, ограниченный ключ, минимальные sudo-права или rootless docker при возможности.
R5	Отсутствие тестов приведет к формальному pipeline без защиты качества.	Высокая/среднее	Минимальные Django tests для / и /health/, отдельная тестовая БД в CI.
R6	Окружения staging/prod будут расходиться из-за ручных правок .env.	Средняя/среднее	Документировать .env.example, хранить значения в защищенных variables, вести журнал изменений.
R7	Reverse proxy/TLS не покрыт текущим локальным compose.	Средняя/среднее	Ввести docker-compose.prod.yml, TLS/certs, проверку домена и статуса проху в чек-листе.

Эксплуатационный чек-лист после релиза

Периодичность	Действие	Подтверждение
Каждый релиз	Проверить /health/, docker compose ps, последние 100 строк логов web/db, успешность миграций.	Запись в релизном журнале.
Ежедневно	Проверить наличие backup БД и свободное место на диске.	Автоматический отчет или отметка администратора.
Еженедельно	Проверить обновления базовых образов, зависимости Python, очистку старых registry tags.	Issue/MR или отметка по action.
Ежемесячно	Провести тест восстановления backup на отдельном окружении.	Протокол restore rehearsal.
Ежеквартально	Пересмотреть доступы GitLab/SSH, ключи, секреты, домены, TLS.	Акт/запись ревизии доступов.

Критерии приемки внедрения

- CI/CD pipeline запускается из GitLab и содержит минимум: lint/check, test, build, push image, staging deploy, manual production deploy.
- Ни один production-секрет не хранится в репозитории; .env.example содержит только шаблонные значения.
- Docker image версионизируется SHA/tag и доступен из registry; rollback на предыдущий tag задокументирован и проверен.
- Staging и production используют отдельные переменные и отдельные server directories.
- Endpoint /health/ проверяет доступность БД и используется в smoke tests.
- Перед production есть backup БД и процедура восстановления.
- В README_DEPLOY описаны запуск, релиз, откат, диагностика и ответственные.

Источники и проверенные факты

Дата подготовки: 20 июня 2026 г. Источники использованы для инвентаризации репозиторий и обоснования технических решений.

- GitLab: <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service> - публичная карточка проекта: Project ID 399, 29 commits, 2 branches, наличие CI/CD configuration, Container Registry, файл .gitlab-ci.yml 2.86 KB; содержимое .gitlab-ci.yml не было доступно как текст без авторизации/рендера.
- GitHub: <https://github.com/schudnaya/mock-django> - публичный репозиторий mock-django, 10 commits, файлы app, Dockerfile, README.md, docker-compose.yml, requirements.txt; README содержит локальный запуск через docker compose up -d --build и переход на <http://localhost:8080>.
- Dockerfile mock-django: Python 3.11 Alpine, установка системных пакетов, pip install -r requirements.txt, запуск Gunicorn config.wsgi:application на 0.0.0.0:8000.
- docker-compose.yml mock-django: сервисы web, db(PostgreSQL 15 Alpine), nginx-proxy; порт хоста 8080 -> 80; переменные POSTGRES_* заданы в compose.
- Django settings.py mock-django: SECRET_KEY задан в коде, DEBUG=False, ALLOWED_HOSTS=["*"], подключение к PostgreSQL через переменные окружения; URL /health/ выполняет SELECT 1 к БД.
- GitLab Docs: .gitlab-ci.yml описывает jobs pipeline; predefined variables доступны в каждом pipeline; CI/CD variables применяются для переиспользования значений и предотвращения hard-code секретов; GitLab CI/CD может строить, тестировать и push Docker images.
- Docker Docs: Compose создает сервисы в порядке зависимостей; healthcheck/depends_on с condition: service_healthy позволяют ждать готовности зависимого сервиса.
- Django Docs: deployment checklist рекомендует manage.py check --deploy, секретный SECRET_KEY вне source control, DEBUG=False в production, конкретные ALLOWED_HOSTS, STATIC_ROOT/collectstatic, HTTPS и secure cookies.